# Kaizen — Documentación técnica

Guía de arquitectura para entender, mantener o **replicar** la app (p. ej. describiéndosela a otra IA). Última actualización: octubre 2026.

## 1. Qué es

Kaizen — *Iteration Design Manager* es una **app de una sola página** para diseñar juegos de mesa: ideas e iteraciones, reglamento versionado, prototipo jugable, "Board" de notas, tareas pendientes, preparación de **ferias** (citas con editoriales) y **seguimiento de editoriales** (CRM tipo tracking). Funciona en el navegador y como **app instalable (PWA) con modo offline**.

## 2. Stack y decisiones

- **Front:** un único archivo HTML con CSS y JavaScript *vanilla* embebidos (sin framework, sin bundler, sin build de JS). Tema oscuro, variables CSS.
- **Backend/datos:** Supabase (Postgres + Auth + RLS + Edge Functions). La clave *anon* es pública y va en `config.js`; la seguridad la dan las políticas **RLS**.
- **Hosting:** Vercel, redeploy automático en cada commit de GitHub. La web solo necesita `index.html` + `config.js` (+ los archivos PWA).
- **PWA:** `manifest.webmanifest` + `sw.js` (service worker) para instalar y abrir sin conexión.
- **Sin dependencias de build.** La única librería externa es `@supabase/supabase-js@2` desde CDN (jsDelivr).

## 3. Estructura del repositorio (carpeta publicada `iteraciones-app/`)

Lo que la web necesita, en la raíz de la carpeta publicada: - **`index.html`** — la aplicación (generada, ver §4). - **`config.js`** — `window.SUPABASE_CONFIG = { url, anonKey }`. Si está vacío, la app funciona en modo local (localStorage) sin nube. - **`sw.js`** — service worker (offline + autoactualización). - **`manifest.webmanifest`** + `icon-192.png`, `icon-512.png`, `icon-maskable-512.png`, `apple-touch-icon.png`.

Material de trabajo / documentación (no lo usa la web): - **`iteraciones.html`** (en la raíz del proyecto, un nivel por encima) — **archivo fuente** que se edita. - **`build.py`** — genera `iteraciones-app/index.html` desde `iteraciones.html`. - **`docs/`** — esta documentación, SQL y funciones Edge.

## 4. Flujo de publicación (build)

El código fuente vive en `iteraciones.html`. `build.py` lo copia a `iteraciones-app/index.html` **inyectando antes de `</head>`**:

```
<script src="https://cdn.jsdelivr.net/npm/@supabase/supabase-js@2"></script>
<script src="config.js"></script>
<script>/* preconnect dinámico al dominio de Supabase leyendo config.js */</script>
```

Para publicar: ejecutar `python build.py`, subir `index.html` (y, si cambian, `sw.js` / `config.js` / iconos) al repo. Vercel redepliega. Validación recomendada antes de publicar: extraer el mayor `<script>` y pasar `node --check`.

## 5. Capa de almacenamiento: `Store`

Objeto `Store` que abstrae el backend. `Store.backend` ∈ `local` | `cowork` | `supabase` | `supabase-anon`.

- `init()` — crea el cliente Supabase si hay `SUPABASE_CONFIG` y sesión; si no, `local`.
- `get(key)` / `set(key,val)` — en supabase leen/escriben la fila del usuario en `app_state`; en local usan `localStorage`.
- Juegos compartidos: `listSharedGames()`, `createSharedGame(data)`, `updateSharedGame(id,data,rev)` (bloqueo optimista por `rev`), `forceUpdateSharedGame(id,data)` (última versión gana), `deleteSharedGame(id)`, `leaveGame(id)`.
- Perfiles/identidad: `upsertProfile()`, `setUsername()`, `changeName()`, directorio `listAllProfiles()`.
- Admin/solicitudes: `checkAdmin()`, `listSignupRequests()`, anuncios, invitaciones, muro.

## 6. Modelo de datos (objeto `state`, serializado a JSON)

```
state = {
  games: [ Game ],          // juegos propios del usuario
  generalTasks: [ Task ],   // tareas sin juego ("General"), solo en Inicio
  fairs: [ Fair ],          // ferias
  editorials: [ EdEntry ],  // seguimientos juego↔editorial (CRM)
  currentGameId, preloadVersion, updatedAt
}
sharedGames = [ Game + meta __ ]   // en memoria, cargados de la nube (no en state)
```

**Game**

```
{ id, name, color, stage: 'active'|'draft'|'paused', touchedAt,
  changes: [ Idea ], tasks: [ Task ], posts: [ Post ],
  rulebook: { sections:[{key,title,content}] }, rulebookVersions:
[{n,sections,summary,...}],
  proto: { name, html, updatedAt } }
// Compartido añade meta: __shared, __sharedId, __rev, __owner, __members[],
__updatedBy, __updatedByName, __updatedAt
```

**Idea (change)** — pestaña Ideas:

```
{ id, title, idea, note,
state:'porprobar'|'aparcada'(Testing)|'funciona'(OK)|'descartada',
  section, implemented, implementedSection, implementedSig, appImpact,
appImplemented, createdAt, updatedAt }
```

**Task** (en `game.tasks` o `state.generalTasks`):

```
{ id, text, done, createdAt, doneAt, editorial? }   // 'editorial' etiqueta las
tareas automáticas de envío
```

**Post** (Board): `{ id, by, byName, text, images:[dataURL], createdAt, updatedAt, edited }`

**Fair**: `{ id, name, start, end, dates, place, past, appointments:[ Appt ] }`

**Appt** (cita): `{ id, editorial, when, place, contact, done, doneAt, games:[ ApptGame ] }`

**ApptGame** (juego mostrado en una cita):

```
{ gameId, gameName, estado:'pendiente'|'proto_dejado'|'interesado'|'descartado',
  follow:[ 'proto'|'pnp'|'reglas'|'video'|'sellsheet'|'agendar' ],  // solo si
interesado/proto_dejado
  feedback, taskId, taskAgendarId }   // ids de las tareas automáticas creadas
```

**EdEntry** (seguimiento editorial, CRM):

```
{ id, editorial, gameId, gameName,
status:'interesado'|'proto_dejado'|'testing'|'firmado'|'descartado',
  createdAt, updates:[ { id, text, createdAt, dir:'out'|'in'|'note', fromFair? } ] }
```

El `log` (`updates`) es tipo conversación: `dir` = *Yo→editorial* / *Editorial→yo* / *Nota*. La primera línea suele venir de la cita de feria (`fromFair:true`).

## 7. Relaciones automáticas (reglas de negocio)

- En una cita, marcar un juego como **Interesado** o **Proto dejado** → crea/asegura un **EdEntry** (con la editorial de la cita) y siembra la primera línea del log desde la cita. **Descartado** también crea EdEntry con estado `descartado`.
- En **Interesado/Proto dejado** se despliegan casillas de seguimiento. Marcar envíos (`proto/pnp/reglas/video/sellsheet`) crea/actualiza **una** tarea en el juego: `"Enviar PNP + Reglas a {Editorial}"`. `agendar` crea otra tarea `"Agendar TTS a {Editorial}"`. En Proto dejado se oculta "Enviar proto". Esas tareas se etiquetan con `editorial` y aparecen también en la pestaña Tareas del juego y en Inicio.
- Al **guardar la cita** y **al cargar** se re-aplica la editorial a esas tareas (por si se escribió el nombre después).
- El **nombre del juego** se resuelve siempre en vivo por `gameId` (si renombras un juego, se actualiza en ferias/editoriales).

## 8. Persistencia y modo offline

- `persistState()` escribe en la nube **y** en una **copia local** ( `localStorage` key `kaizen-cache-<userId>` ), con bandera `kaizen-dirty` si no pudo subir.
- `loadState()` : carga primero la copia local (apertura instantánea); intenta la nube con **timeout** ( `withTimeout` , 5 s); si hay cambios locales sin sincronizar más nuevos, gana lo local; si no, gana la nube. Offline → usa la copia local.
- **Compartidos offline:** se cachean ( `kaizen-shared-<userId>` ); edición offline se encola ( `kaizen-shared-pending-<userId>` ) y al reconectar pregunta si sincronizar (**última versión gana** ante conflicto).
- **Reanudar:** al volver la app a primer plano ( `visibilitychange` / `focus` / `pageshow` ) o recuperar conexión ( `online` ), se vuelve a traer la nube y repinta (clave en iOS, que *reanuda* la PWA en vez de recargar).
- **Service worker ( `sw.js` ):** mismo origen = *stale-while-revalidate* (abre al instante, refresca en segundo plano); CDN = caché primero; llamadas a `*.supabase.co` = nunca se cachean. Autoactualiza sin versiones.

## 9. Render y navegación

- `appView` ∈ `home` | `design` | `ferias` | `editoriales` | `fair` | `game` . `render()` enruta a `renderHomeHub` , `renderDesignPortada` , `renderFeriasPortada` , `renderEditorialesPortada` , `renderFairBoard` , `renderBoard` .
- Las "portadas" (home/design/ferias/editoriales) se pintan en `#portada` ; la vista de juego/feria en `#wrap` dentro de `.content` .
- `saveState()` enruta: si estás en un juego compartido, guarda en la nube ( `saveSharedGame` ); si no, `persistState()` . Helper `persistGame(g)` para guardar un juego concreto (propio o compartido).
- Normalización/migraciones: `normalizeStructures()` adapta datos antiguos al modelo nuevo y hace *backfills* (p. ej. crear seguimientos de editoriales desde citas ya marcadas).

## 10. Supabase: tablas y funciones (ver `docs/sql` y `docs/edge` )

Orden de ejecución del SQL (SQL Editor): 1. `supabase-schema.sql` — `app_state` (estado por usuario) + `signup_requests` . 2. `shared-games-schema.sql` — `profiles` (@usuario), `shared_games` , `game_invites` + función de aceptar. 3. `admins-schema.sql` — `admins` + restringe solicitudes a admins. 4. `leave-game-fix.sql` — función para salir de un compartido. 5. `announcements-schema.sql` — anuncios del admin. 6. `webhook-trigger.sql` — (opcional) dispara email al llegar una solicitud.

Edge Functions (Supabase → Edge Functions, *Verify JWT off* donde se indica): `notify-signup` , `approve-signup` , `admin-users` . Correos: ver `docs/email` .

Seguridad: todo el acceso a datos pasa por **RLS**; un usuario solo ve/edita su `app_state` , los `shared_games` donde es dueño o miembro, su perfil y lo compartido con él.

## 11. Cómo replicarlo en otro chat de IA (resumen-prompt)

> Crea una SPA en **un solo archivo HTML** (CSS+JS vanilla, tema oscuro con variables CSS, sin frameworks ni build) para gestionar el diseño iterativo de juegos de mesa. Debe tener: un objeto `Store` que abstraiga el backend (localStorage o Supabase según `window.SUPABASE_CONFIG`); un `state` con `games[]`, `generalTasks[]`, `fairs[]`, `editorials[]`; navegación por `appView` con una función `render()`. Un juego tiene pestañas **Ideas** (con estados Por probar/Testing/OK/Descartada, "llevar al reglamento" y "APP idea"), **Tareas**, **Board** (notas con imágenes y dibujo a mano en canvas), **Reglas** (reglamento versionado) y **Proto** (HTML jugable embebido en iframe). Inicio = dashboard (métricas, "Idea rápida", "Tarea rápida", áreas Diseño/Ferias/Editoriales, tareas e ideas pendientes). **Ferias**: citas con editorial, fecha, lugar; cada cita lista juegos con estado (Pendiente/Proto dejado/Interesado/Descartado); Interesado/Proto dejado despliega casillas de seguimiento que generan tareas automáticas y crean entradas de **Editoriales**. **Editoriales (Tracking)**: por juego, pestaña *Activos* (kanban de columnas Interesado/Proto dejado/En Testing) y *Descartados* (lista con último feedback); cada seguimiento tiene un **log** de iteraciones con dirección (Yo→editorial / Editorial→yo / Nota). Persistencia con copia local + nube y modo **offline** (service worker + cache de estado). Juegos **compartidos** multiusuario con bloqueo optimista por `rev`. Instalable como **PWA**.

Con esa descripción y el modelo de datos de §6–§7, otra IA puede reconstruir la app.